



Teste de Software

Prof. Dr. Bruno Queiroz Pinto

Testes Automatizados em Java

Estrutura de Teste

- Cenário de Teste:
 - monta o cenário,
 - executa a ação
 - e valida a saída.

BeforeEach

- Nos códigos desenvolvidos até agora há muita repetição de código.
 - Instanciação de classes. Imagina se modificarmos o construtor.... Problemão...

Solução:

- Definir esses objetos como atributos da classe de Teste.
- Criar um método para instanciar essas classes.
- Chamar o método de instanciação sempre que preciso.

@BeforeEach

BeforeEach

A notação @BeforeEach força a execução do método antes da execução de cada um dos testes.

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
```

```
public class CalculadoraTest {
    private Calculadora calc;
```

```
    @BeforeEach
    public void inicializaCenario() {
        // instanciar objetos
        calc = new Calculadora();
    }
```

Se você importar Test de
org.junit.Test
dará erro

Código sempre será executado
antes de cada teste

Ou seja, a cada novo teste um objeto calculadora será criado

AfterEach, AfterAll, BeforeAll

Métodos anotados com `@AfterEach` são executados após a execução do método de teste. Usamos métodos `@AfterEach` quando nossos testes consomem recursos que precisam ser finalizados. Exemplos podem ser testes que acessam banco de dados, abrem arquivos, abrem sockets etc.

Métodos anotados com `@BeforeAll` e `@AfterAll` são executados apenas uma vez, antes e depois de todos os métodos de teste, respectivamente. **São métodos estáticos.**

No caso da calculadora poderíamos utilizar o `BeforeAll`

Necessário definir como `static` o método e o atributo

Testando Exceptions

Em nosso código da calculadora, devemos modificar o método dividir para retornar uma exception no caso do número divisor igual a 0

```
public int dividir(int num1, int num2) throws Exception {  
    if (num2 == 0)  
        throw new Exception("Divisão por zero!!!");  
    return num1/num2;  
}
```

Retorna uma exception

Testando Exceptions : novo teste

```
@Test
public void testarDivisaoPorZeroRetornaErro() {
    try {
        calculadora.dividir(numero1, numero2);
    } catch (Exception e) {
        // tudo ok;
    }
}
```

Código simples - ruim

```
@Test(expected=RuntimeException.class)
public void testarDivisaoPorZeroRetornaErro() {
    calculadora.dividir(numero1, numero2);
}
```

Código Junit 4

```
@Test
public void testarDivisaoPorZeroRetornaErro() {
    assertThrows(Exception.class,
        ()->calculadora.dividir(numero1, numero2));
}
```

Código Junit 5

Outros assert...

```
@Test
public void teste() {
    assertEquals(10, 10);
    assertEquals(10.0, 10.0001, 0.01, "");
    assertEquals(new String("OI"), new String("OI")); // método equals
    assertNotEquals(10, 11); // inverso do assertEquals
    assertFalse(false);
    assertTrue(true);
    assertNotNull(calculadora);
    assertNull(null);
    assertSame(calculadora, calculadora); // mesma referência
    assertNotSame(new String("OI"), new String("OI"));
    assertEquals(new int[0], new int[0]);
    assertThrows(Exception.class, () ->{throw new Exception("Teste");});
    assertDoesNotThrow(() ->{int a=10;});
    assertLinesMatch(new ArrayList<String>(), new ArrayList<String>());
    // testar o tempo de execução, muito útil
    assertTimeout(java.time.Duration.ofSeconds(2), () ->{});
    // testar se todas as funções não retornam exception
    assertAll(() ->{int a=10;}, () ->{int a=10;}, () ->calc.dividir(2,3));
    //fail("Teste não implementado"); //faz o teste falhar

    // inserir um novo atributo String com uma mensagem
    // personalizada de erro
    assertEquals(10, 11, "Erro : números diferentes");
}
```


AssertTimeout

```
@Test
public void teste1() {
    assertTimeout(java.time.Duration.ofMillis(1), ()->{
        for (int i = 0; i < 10000; i++) {
            int x = i;
        }
    }, "Tempo limite ultrapassado");
}

@Test
public void teste2() {
    assertTimeout(java.time.Duration.ofNanos(1),
        ()->{calculadora.dividir(10,5);}
        , "Tempo limite ultrapassado");
}
```